



Zoom through C
unit (W 2/4)
In-person +
recording only
starting F 2/6

CS61C

**Great Ideas
in
Computer Architecture**
(a.k.a. Machine Structures)



Assistant Teaching
Professor
Lisa Yan

C: Odds and Ends



Working with Memory



Code, Static storage are easy:

- They never grow or shrink.

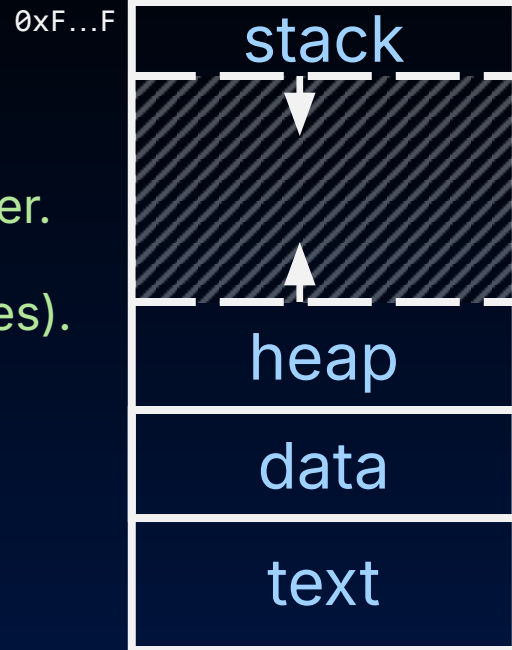


Stack space is also "easy":

- Stack frames are created/destroyed in LIFO order.
- Just avoid **dangling references**: pointers to deallocated variables (e.g., from old stack frames).



Working with the heap is **tricky**.





The Heap

- The Heap
- Faulty Heap Management
- Words, Endianness
- [at home] Linked List Example
- [at home] Bitwise Operations

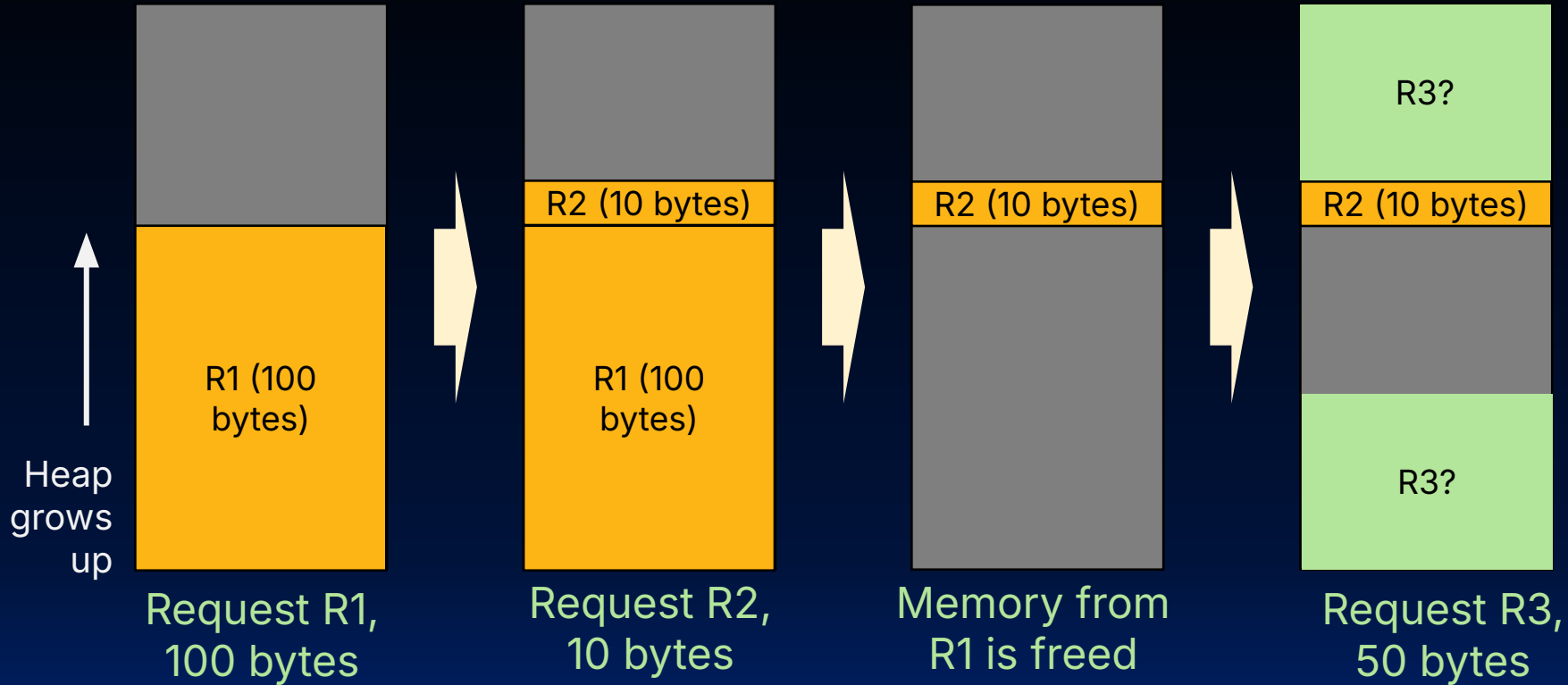


What is the Heap?

- The heap is **dynamic memory**: can be allocated, resized, and freed during program runtime.
 - Useful for persistent memory across function calls.
 - Similar to Java new command, which allocates memory...
 - ...but key differences lead to biggest source of C memory leaks/bugs.
- Huge pool of memory (usually >> stack) **not** allocated in contiguous order.



Heap Management Example



depends on heap allocator strategy
Yan, SP26



stdlib.h heap

- C heap standard library functions: specify # bytes of memory explicitly to allocate/reallocate.
 - `malloc()`: Allocates raw, uninitialized memory from heap
 - `free()`: Frees memory on heap
 - `realloc()`: Resizes previously allocated heap blocks to new size

Unlike the stack, heap memory gets reused only when programmer explicitly cleans up.



stdlib.h heap: See Manual

MALLOC(3)

Linux Programmer's Manual

MALLOC(3)

NAME

malloc, free, calloc, realloc, reallocarray - allocate and free dynamic memory

SYNOPSIS

```
#include <stdlib.h>
```

```
void *malloc(size_t size);
```

```
void free(void *ptr);
```

```
void *calloc(size_t nmemb, size_t size);
```

```
void *realloc(void *ptr, size_t size);
```

```
void *reallocarray(void *ptr, size_t nmemb, size_t size);
```

Pro Tip: run **man malloc** on hive! (hit q to exit)
The Linux manual page describes the heap API
(and undefined behavior).

Let's briefly discuss
stdlib.h heap API

void *malloc(size_t n)

- Allocates a block of uninitialized memory:
 - **size_t n**: unsigned integer type big enough to “count” memory bytes.
 - Returns **void *** pointer to block of memory on heap.
 - A return of **NULL** indicates no more memory (always check for it!!!)

- To allocate, say, a struct:

```
typedef struct { ... } TreeNode;  
TreeNode *tp = malloc(sizeof(TreeNode));
```

size of (type): size in bytes.

- To allocate an array of 20 ints:

```
int *ptr = malloc(20*sizeof(int));  
  
int ptr[] = malloc(20*sizeof(int));
```

Assuming size of objects can lead to misleading, unportable code. Use `sizeof()`!

void free(void *ptr)

- Dynamically frees heap memory
 - `ptr` is a pointer containing an address originally returned by `malloc()/realloc()`.

```
int *ptr = (int *) malloc (sizeof(int)*20);  
...  
free(ptr); // implicit typecast to (void *)
```

- When you free memory, be sure to pass the **original address** returned from `malloc()`. Otherwise, crash (or worse!)

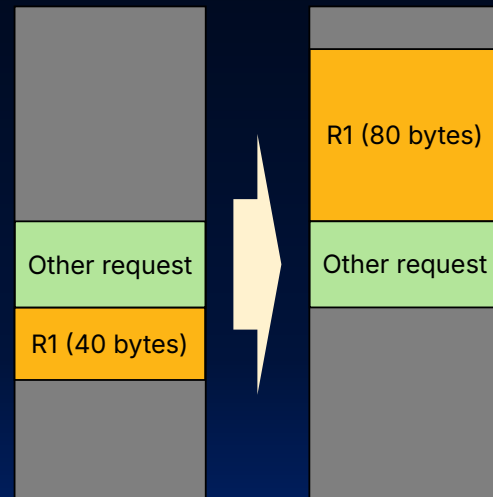


void *realloc(void *ptr, size_t size)

- Resize a previously allocated block at ptr to a new size.
 - Returns new address of the memory block.
 - In doing so, it may need to copy all data to a new location.

```
int *ip = malloc(10*sizeof(int));  
... ..  
ip = realloc(ip, 20*sizeof(int));  
/* contents of first  
   10 elements retained */  
... ..
```

- Remember: Always check if returned pointer is NULL—which would mean you've run out of memory!



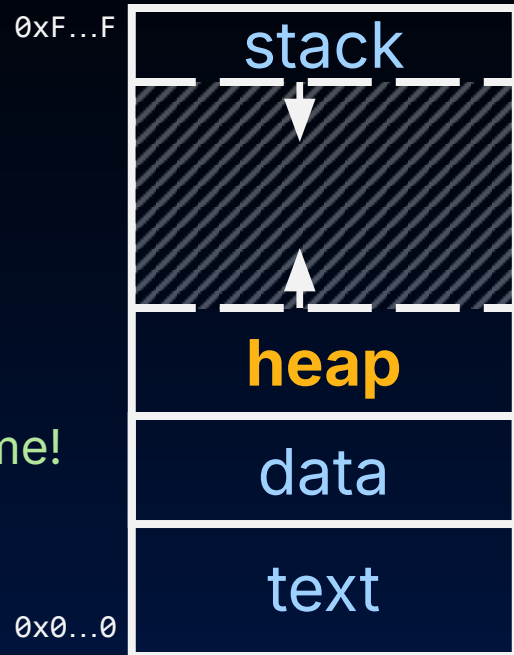


Faulty Heap Management

- The Heap
- Faulty Heap Management
- Words, Endianness
- [at home] Linked List Example
- [at home] Bitwise Operations

Working with Memory

- ⚠ Working with the heap is **tricky**.
- Memory can be allocated / deallocated at any time!
 - **Memory leak**: You forget to deallocate memory } Will eventually run out of memory
 - **"Use after free"**: (self-explanatory) }
 - **"Double free"**: Call free 2x on same memory } Possible crash or exploitable vulnerability





Faulty Heap Management

How many memory management errors are in this code?

```
void free_mem_x() {  
    int fnh[3];  
    ...  
    free(fnh);  
}
```

```
void free_mem_y() {  
    int *fum = malloc(4*sizeof(int));  
    free(fum+1);  
    ...  
    free(fum);  
    ...  
    free(fum);  
}
```

- A.** 1
- B.** 2
- C.** 3
- D.** 0
- E.** Other

L05: How many memory management errors are in this code?

```
void free_mem_x() {  
    int fnh[3];  
    ...  
    free(fnh);  
}  
  
void free_mem_y() {  
    int *fum = malloc(4*sizeof(int));  
    free(fum+1);  
    ...  
    free(fum);  
    ...  
    free(fum);  
}
```

A. 1

0

B. 2

0

C. 3

0

D. 0

0

E. Other

0





Faulty Heap Management

How many memory management errors are in this code?

```
void free_mem_x() {  
    int fnh[3];  
    ...  
    free(fnh);  
}
```

} free() on stack-allocated memory

```
void free_mem_y() {  
    int *fum = malloc(4*sizeof(int));  
    free(fum+1);  
    ...  
    free(fum);  
    ...  
    free(fum);  
}
```

} free() on memory that isn't the pointer from malloc

} Double free()

- A. 1
- B. 2
- C. 3**
- D. 0
- E. Other



Valgrind to the rescue!!!

- Valgrind slows down your program by an order of magnitude, but...
- Valgrind adds a tons of checks designed to catch most (but not all) memory errors
 - Memory leaks
 - Misuse of free
 - Writing over the end of arrays
- Tools like Valgrind are absolutely essential for debugging C code.

Check out Lab 02!



Words, Endianness

- The Heap
- Faulty Heap Management
- Words, Endianness
- [at home] Linked List Example
- [at home] Bitwise Operations



Word Size

2^{32}

An architecture's **word size** determines many aspects of computer operation.

- def **word size**: natural unit of access in a computer
 - the smallest accessible unit of memory
 - the size of an address
 - etc.
- An N-bit architecture by convention has N-bit words and therefore N bits in an address.
 - Common N: 32-bit, 64-bit

	+3	+2	+1	+0
0x0	xx	xx	xx	xx
0x4	xx	xx	xx	xx
0x8	xx	xx	xx	xx
0xC	xx	xx	xx	xx
0x10	xx	xx	xx	xx
...	xx	xx	xx	xx
...	xx	xx	xx	xx
0xFFFFFFFF0	xx	xx	xx	xx
0xFFFFFFFF4	int32_t *			
0xFFFFFFFF8	char *			
0xFFFFFFF0C	xx	xx	xx	xx

Yan, SP26



0x...02

0x...00

- This view of memory closely reflects how gdb prints out program memory, from low addresses to higher addresses.
- Groups by **words**
- Assumes **little endianness** (?!)

	+3	+2	+1	+0
0x0	xx	xx	xx	xx
0x4	xx	xx	xx	xx
0x8	xx	xx	xx	xx
0xC	xx	xx	xx	xx
0x10	xx	xx	xx	xx
...	xx	xx	xx	xx
...	xx	xx	xx	xx
0xFFFFFFFF0	xx	xx	xx	xx
0xFFFFFFFF4	int32_t *			
0xFFFFFFFF8	char *			
0xFFFFFFF0	xx	xx	xx	xx

[high mem on bottom] →



Example program memory, on hive

see: endianness.c

```
#include <stdio.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    int32_t value = 0x12345678;
    char str[] = "bye";
    char str2[] = "hello";
    int16_t short_val = 0xaabb;
    ...

    return 0;
}
```

	+3	+2	+1	+0
0x0	xx	xx	xx	xx
0x4	xx	xx	xx	xx
...	xx	xx	xx	xx
0x7F...FE164	0xaa	0xbb	xx	xx
0x7F...FE168	0x12	0x34	0x56	0x78
0x7F...FE16C	'y'	'b'	xx	xx
0x7F...FE170	'e'	'h'	'\0'	'e'
0x7F...FE174	'\0'	'o'	'l'	'l'
0x7F...FE178	xx	xx	xx	xx
...	xx	xx	xx	xx
0xFFFFFFFFFC	xx	xx	xx	xx

Yan, SP26

Endianness

- Endianness: When data occupies multiple bytes, the computer must determine which byte is at the lowest byte address.
- The hive machines are **little endian**.
 - Words are stored **least significant byte (LSB) first**.
 - Diagram makes it easy to read integers, harder to read strings:
`char str[] = "bye";`



	+3	+2	+1	+0
0x0	xx	xx	xx	xx
0x4	xx	xx	xx	xx
...	xx	xx	xx	xx
0x7F...FE164	0xaa	0xbb	xx	xx
0x7F...FE168	0x12	0x34	0x56	0x78
0x7F...FE16C	'y'	'b'	xx	xx
0x7F...FE170	'e'	'h'	'\0'	'e'
0x7F...FE174	'\0'	'o'	'l'	'l'
0x7F...FE178	xx	xx	xx	xx
...	xx	xx	xx	xx
0xFFFFFFF0	xx	xx	xx	xx

For you to try at home with gdb

```
$ gdb endianness
(gdb) b 9 # set breakpoint at line 9
(gdb) r    # run, initializing all vars
(gdb) p/x str # see string bytes
(gdb) p/x str2
(gdb) # print one word, show in hex
(gdb) x/1wx 0x7fffffffef164
(gdb) <enter> # repeat last action
(gdb) ... # keep pressing <enter>
(gdb) # LSB in lowest address/word
(gdb) # print 4 bytes, show in hex
(gdb) x/4bx 0x7fffffffef164
(gdb) <enter> # repeat last action
(gdb) ... # keep pressing <enter>
(gdb) q
```

	+3	+2	+1	+0
0x0	xx	xx	xx	xx
0x4	xx	xx	xx	xx
...	xx	xx	xx	xx
0x7F...FE164	0xaa	0xbb	xx	xx
0x7F...FE168	0x12	0x34	0x56	0x78
0x7F...FE16C	'y'	'b'	xx	xx
0x7F...FE170	'e'	'h'	'\0'	'e'
0x7F...FE174	'\0'	'o'	'l'	'l'
0x7F...FE178	xx	xx	xx	xx
...	xx	xx	xx	xx
0xFFFFFFFFC	xx	xx	xx	xx

Word Alignment

- Many architectures are **word-aligned**:
 - Access words on word boundaries (e.g., 32b on 4-byte boundaries)
 - Some architectures will be slow if you try to access "unaligned" memory
 - Other processors may not even allow you access!
- Compilers will typically figure this out for you.

	+3	+2	+1	+0
0x0	xx	xx	xx	xx
0x4	xx	xx	xx	xx
...	xx	xx	xx	xx
0x7F...FE164	0xaa	0xbb	xx	xx
0x7F...FE168	0x12	0x34	0x56	0x78
0x7F...FE16C	'y'	'b'	xx	xx
0x7F...FE170	'e'	'h'	'\0'	'e'
0x7F...FE174	'\0'	'o'	'l'	'l'
0x7F...FE178	xx	xx	xx	xx
...	xx	xx	xx	xx
0xFFFFFFFFC	xx	xx	xx	xx

Structures, Revisited, Again

A "struct" is really an instruction to C on how to arrange a bunch of bytes in a bucket.

- The precise layout is architecture-dependent!

C compilers often **align** structs with **padding**.

- Example compiled struct on some 32-bit architecture:
 - Note the 3 bytes of padding
 - `sizeof(struct foo) == 12`

```
struct foo {
    int32_t a;
    char b;
    struct foo *c;
}
```

+3	+2	+1	+0
	4 bytes for a		
unused	unused	unused	1 byte for b
	4 bytes for c		



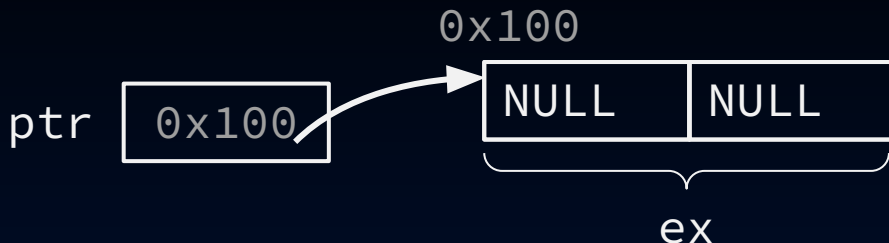
[at home] Linked List Example

Read course notes!

- The Heap
- Faulty Heap Management
- Words, Endianness
- [at home] Linked List Example
- [at home] Bitwise Operations

Struct Pointers and Arrow Notation

```
1 typedef struct _node node_t;
2 struct _node {
3     char *data;
4     node_t *next;
5 };
6
7 node_t ex;
8 node_t *ptr = &ex;
9
10 ex.data = NULL;
11 ptr->next = NULL; // equivalent: (*ptr).next = NULL;
12                  // could also do   ex.next = NULL;
```



"arrow notation"
deref + struct access

Linked List Example

```
# include <string.h>
int main() {
1   node_t *head = NULL;
2   list_add_front(&head, "abc");
   ... // free nodes, strings here...
}
```

```
void list_add_front(node_t **head_ptr, char *data) {
3   node_t *node = (node_t *) malloc(sizeof(node_t));
4   node->data = (char *) malloc(strlen(data) + 1); // extra byte
5   strcpy(node->data, data); // strcpy also copies null terminator
6   node->next = *head_ptr;
7   *head_ptr = node;
}
```

```
typedef struct _node node_t;
struct _node {
    char *data;
    node_t *next;
};
```

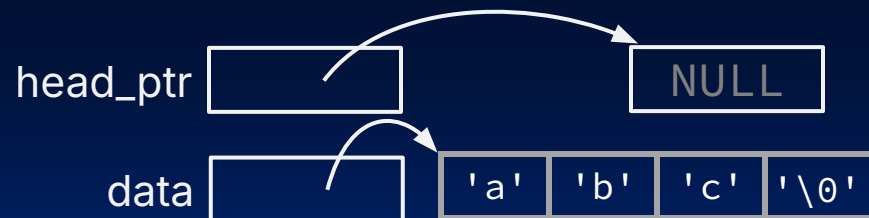
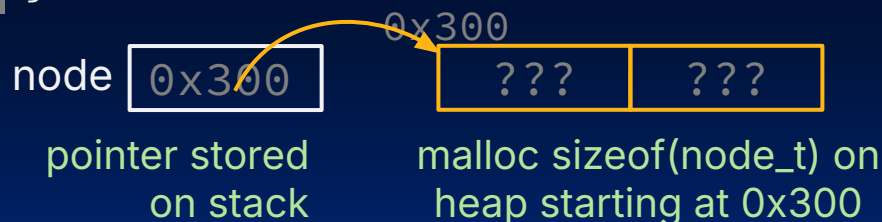


Linked List Example

```
# include <string.h>
int main() {
1   node_t *head = NULL;
2   list_add_front(&head, "abc");
   ... // free nodes, strings here...
}

typedef struct _node node_t;
struct _node {
    char *data;
    node_t *next;
};

void list_add_front(node_t **head_ptr, char *data) {
3   node_t *node = (node_t *) malloc(sizeof(node_t));
4   node->data = (char *) malloc(strlen(data) + 1); // extra byte
5   strcpy(node->data, data); // strcpy also copies null terminator
6   node->next = *head_ptr;
7   *head_ptr = node;
}
```

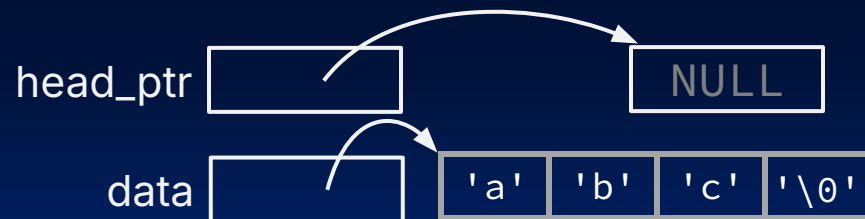


Linked List Example



```
# include <string.h>
int main() {
1   node_t *head = NULL;
2   list_add_front(&head, "abc");
   ... // free nodes, strings here...
}

void list_add_front(node_t **head_ptr, char *data) {
3   node_t *node = (node_t *) malloc(sizeof(node_t));
4   node->data = (char *) malloc(strlen(data) + 1); // extra byte
5   strcpy(node->data, data); // strcpy also copies null terminator
6   node->next = *head_ptr;
7   *head_ptr = node;
}
```



After Line 5 executes, what will the diagram look like?

Linked List Example

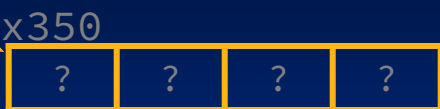
```
# include <string.h>
int main() {
1   node_t *head = NULL;
2   list_add_front(&head, "abc");
   ... // free nodes, strings here...
}

typedef struct _node node_t;
struct _node {
    char *data;
    node_t *next;
};

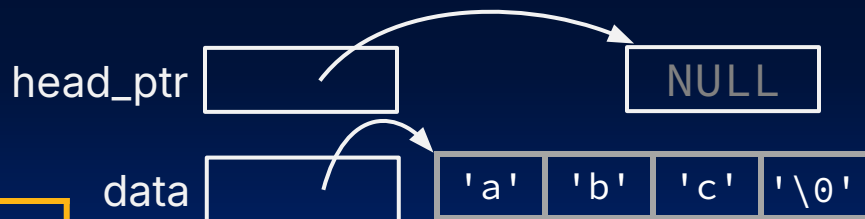
void list_add_front(node_t **head_ptr, char *data) {
3   node_t *node = (node_t *) malloc(sizeof(node_t));
4   node->data = (char *) malloc(strlen(data) + 1); // extra byte
5   strcpy(node->data, data); // strcpy also copies null terminator
6   node->next = *head_ptr;
7   *head_ptr = node;
}
```



malloc 4B on heap
starting at 0x350



06-C Odds and Ends (30)



Linked List Example

```
# include <string.h>
int main() {
1   node_t *head = NULL;
2   list_add_front(&head, "abc");
   ... // free nodes, strings here...
}

typedef struct _node node_t;
struct _node {
    char *data;
    node_t *next;
};

void list_add_front(node_t **head_ptr, char *data) {
3   node_t *node = (node_t *) malloc(sizeof(node_t));
4   node->data = (char *) malloc(strlen(data) + 1); // extra byte
5   strcpy(node->data, data); // strcpy also copies null terminator
6   node->next = *head_ptr;
7   *head_ptr = node;
}
```

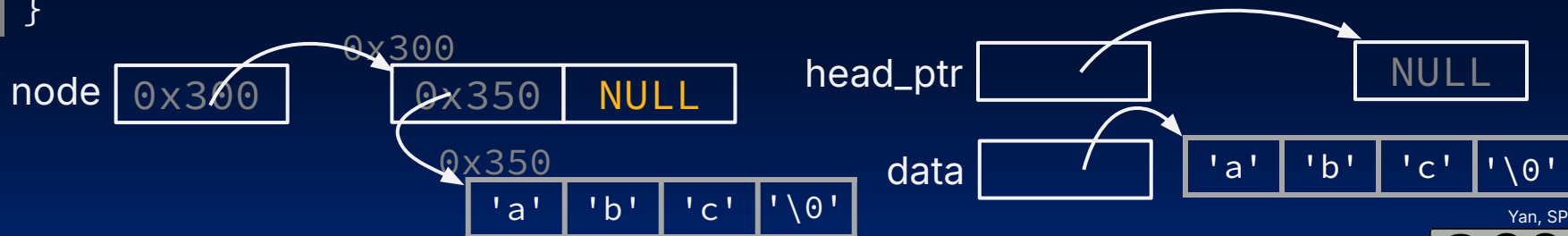


Linked List Example

```
# include <string.h>
int main() {
1   node_t *head = NULL;
2   list_add_front(&head, "abc");
   ... // free nodes, strings here...
}

typedef struct _node node_t;
struct _node {
    char *data;
    node_t *next;
};

void list_add_front(node_t **head_ptr, char *data) {
3   node_t *node = (node_t *) malloc(sizeof(node_t));
4   node->data = (char *) malloc(strlen(data) + 1); // extra byte
5   strcpy(node->data, data); // strcpy also copies null terminator
6   node->next = *head_ptr;
7   *head_ptr = node;
}
```

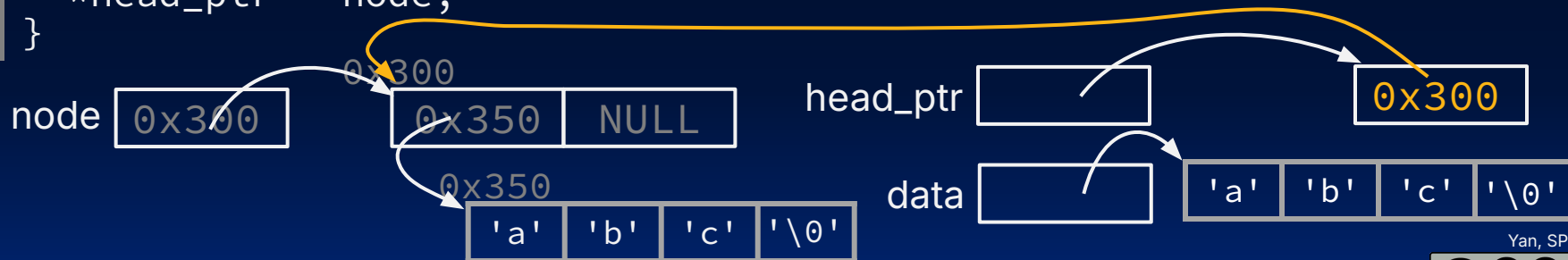


Linked List Example

```
# include <string.h>
int main() {
1   node_t *head = NULL;
2   list_add_front(&head, "abc");
   ... // free nodes, strings here...
}

typedef struct _node node_t;
struct _node {
    char *data;
    node_t *next;
};

void list_add_front(node_t **head_ptr, char *data) {
3   node_t *node = (node_t *) malloc(sizeof(node_t));
4   node->data = (char *) malloc(strlen(data) + 1); // extra byte
5   strcpy(node->data, data); // strcpy also copies null terminator
6   node->next = *head_ptr;
7   *head_ptr = node;
}
```





Implementing a Linked List API in C

- Ideally, want a layer of abstraction:
 - Linked List "API": insert values, pop values, etc.
- In this example, we "broke" a layer of abstraction by allowing main to directly access node_t nodes.
- (See notes for a more detailed implementation)
 - (involves header files, which you will briefly see in Project 1)



Practice with Structs

- The Heap
- Faulty Heap Management
- Words, Endianness
- [at home] Linked List Example
- [at home] Bitwise Operations

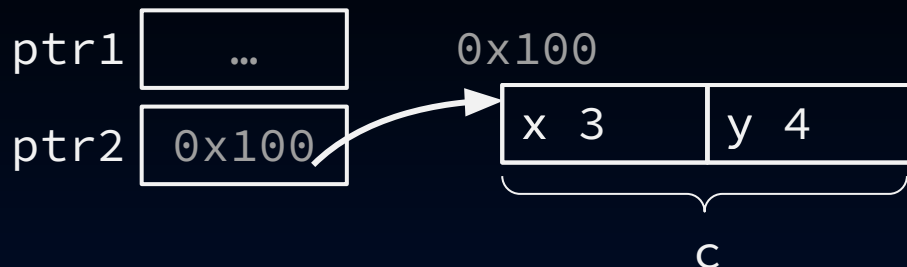
Struct Pointers

```
1 typedef struct {  
2     int x;  
3     int y;  
4 } coord_t;
```

```
5 coord_t c;  
6 coord_t *ptr2 = &c;
```

```
7 c.x = 3;  
8 ptr2->y = 4; // equivalent: *(ptr2).y = 4;
```

```
9 /* This compiles, but what does it do? */  
10 coord_t *ptr1 = ptr2;
```

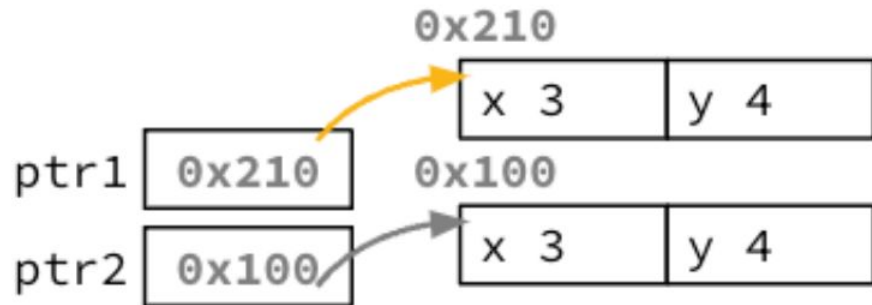


"arrow notation"
deref + struct access

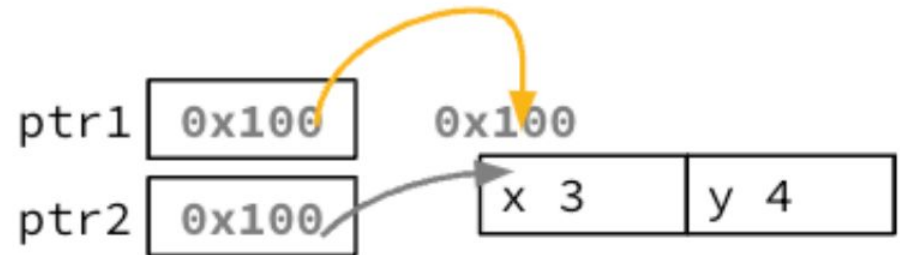
Assume ptr2 points to an initialized Coord struct {3, 4}. What does ptr1 = ptr2; do?



CHOICE A
(click on this side)

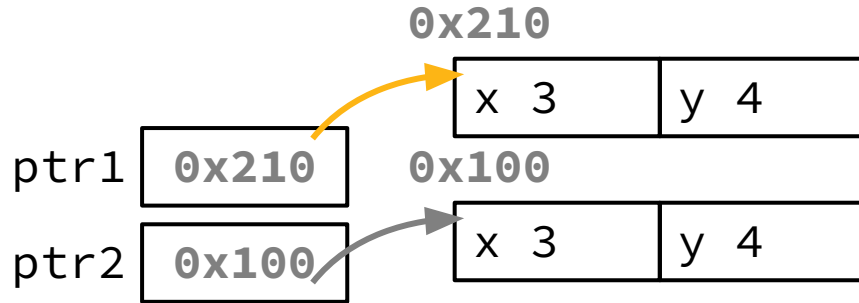


CHOICE B
(click on this side)

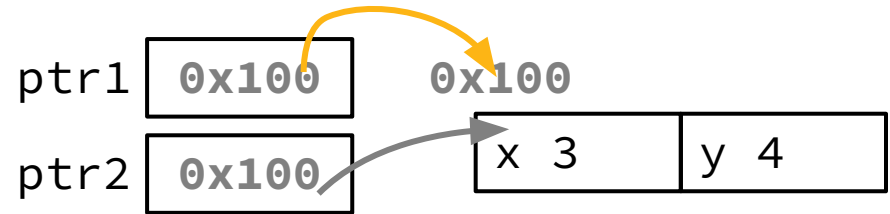




CHOICE A
(click on this side)

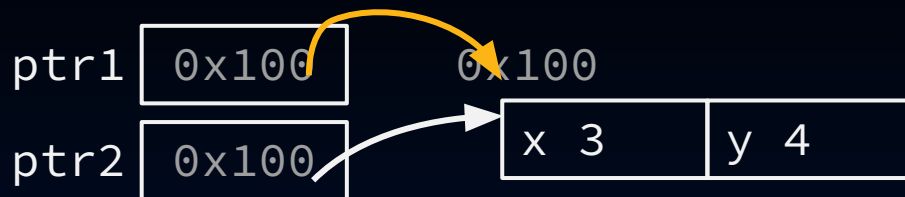


CHOICE B
(click on this side)



Struct Pointers, cont.

```
1 typedef struct {  
2     int x;  
3     int y;  
4 } coord_t;  
  
5 coord_t c;  
6 coord_t *ptr2 = &c;  
  
7 c.x = 3;  
8 ptr2->y = 4; // equivalent: *(ptr2).y = 4;  
  
9 /* This compiles, but what does it do? */  
10 coord_t *ptr1 = ptr2;
```





Agenda

[at home] Bitwise Operations

Read course notes!

- The Heap
- Faulty Heap Management
- Words, Endianness
- [at home] Linked List Example
- [at home] Bitwise Operations

And in Conclusion...

- Bitwise operations: `&`, `|`, `^`, `~` for AND, OR, XOR, NOT
 - Operate on all bits at once
 - `&` is used for masking (keeping some, setting others to 0)
 - `|` is used for keeping some, setting others to 1
 - `^` is used for keeping some, inverting others
- Shifting bits left and right: `<<` and `>>`
- APIs can be created using structs cleanly!
 - Headers in `.h` file specify the API, watch your naming
 - *Implement* them in `.c` files (include the `.h` file)
 - *Use* them in other `.c` files (also include the `.h` file)